

Relatório Técnico

Gerenciador de Tabela de Símbolos com Escopos Aninhados

Disciplina de Compiladores

Introdução

A tabela de símbolos é uma das estruturas de dados mais fundamentais na construção de compiladores e interpretadores. Ela atua como um repositório central que armazena informações sobre os identificadores declarados em um programa-fonte, como variáveis, funções e parâmetros, registrando atributos como nome, tipo, escopo e posição de memória. O analisador léxico, o analisador sintático e, principalmente, o analisador semântico interagem continuamente com essa estrutura durante o processo de compilação.

Em linguagens que suportam escopo léxico — como C, Java e Python — é necessário que a tabela de símbolos seja capaz de gerenciar múltiplos escopos aninhados simultaneamente. Isso significa que uma variável declarada dentro de um bloco (função, laço ou condicional) é visível apenas naquele contexto, e que identificadores de escopos externos podem ser sobrestos por declarações locais.

O presente trabalho descreve a implementação de um gerenciador interativo de tabela de símbolos desenvolvido em Python. O programa simula o comportamento de um compilador real no que diz respeito ao controle de escopos: permite declarar variáveis com nome e tipo, buscar variáveis respeitando a hierarquia de escopos, abrir e fechar escopos dinamicamente, e visualizar o estado atual da pilha de escopos.

Metodologia de Implementação

A implementação foi realizada integralmente em Python 3.11, sem dependências externas, utilizando apenas recursos da biblioteca padrão da linguagem. A arquitetura do projeto é composta por uma classe principal e uma interface de linha de comando interativa.

Estrutura de Dados: Pilha de Escopos

A estrutura central utilizada é uma **pilha de dicionários** (*list of dicts*). Cada dicionário representa um escopo, onde as chaves são os nomes das variáveis e os valores são seus respectivos tipos. A pilha é representada por uma lista Python (`pilha_escopos`), e o escopo ativo é sempre o elemento do topo (último índice). O escopo global é inicializado automaticamente na criação do objeto.

```
class GerenciadorTabelaSimbolos:
    def __init__(self):
        self.pilha_escopos = [{}] # Escopo global inicializado
```

Figura 1 — Inicialização da pilha de escopos com o escopo global.

Operações Implementadas

A classe `GerenciadorTabelaSimbolos` expõe as seguintes operações:

Operação	Descrição	Complexidade
<code>entrar_escopo()</code>	Empilha um novo dicionário vazio na pilha.	$O(1)$
<code>sair_escopo()</code>	Desempilha o topo, descartando variáveis locais. Protege o escopo global.	$O(1)$
<code>declarar(var, tipo)</code>	Insere a variável no escopo do topo. Detecta redeclaração no mesmo escopo.	$O(1)$
<code>buscar(var)</code>	Percorre a pilha do topo à base, retornando o tipo da primeira ocorrência encontrada.	$O(n)$
<code>exibir_pilha()</code>	Imprime todos os escopos da pilha com seus identificadores e tipos.	$O(n)$

Tabela 1 — Operações da classe `GerenciadorTabelaSimbolos`.

Algoritmo de Busca por Escopos

A busca de variáveis implementa a semântica de escopo léxico estático: percorre a pilha do topo (escopo mais local) em direção à base (escopo global), retornando o tipo do primeiro identificador encontrado. Isso garante que uma variável local com o mesmo nome de uma variável global sobrescreva a global no escopo atual.

```
def buscar(self, variavel):
    for escopo in reversed(self.pilha_escopos):
        if variavel in escopo:
            return escopo[variavel]
    return None # Nao declarada em nenhum escopo visivel
```

Figura 2 — Algoritmo de busca que respeita a hierarquia de escopos.

Interface de Linha de Comando

O programa oferece um menu interativo com seis opções de comando, aceitando tanto entrada numérica quanto palavras-chave textuais (ex: 1 ou `declarar`). A função `main()` realiza o parsing da entrada do usuário e despacha para o método correspondente da classe.

Casos de Teste

Os casos de teste a seguir validam os comportamentos esperados do gerenciador de tabela de símbolos, cobrindo declarações simples, controle de escopos aninhados, detecção de erros semânticos e busca hierárquica.

Caso de Teste 1 — Declaração e busca simples

Entrada:

```
opcao: 1
nome e tipo: x int
opcao: 2
variavel: x
opcao: 5
```

Saída esperada:

```
[SUCESSO] Declarado: 'x' do tipo 'int' no escopo atual.
[SUCESSO] Variavel 'x' encontrada! Tipo: 'int'
```

```
--- ESTADO ATUAL DA PILHA DE ESCOPOS ---
Global: {'x': 'int'}
-----
```

Resultado: PASSOU. A variável foi declarada e localizada corretamente no escopo global.

Caso de Teste 2 — Redecaração no mesmo escopo (erro semântico)

Entrada:

```
opcao: 1
nome e tipo: y float
opcao: 1
nome e tipo: y string
```

Saída esperada:

```
[SUCESSO] Declarado: 'y' do tipo 'float' no escopo atual.
[ERRO SEMANTICO] Variavel 'y' ja declarada neste escopo.
```

Resultado: PASSOU. O sistema detectou e reportou a redeclaração corretamente.

Caso de Teste 3 — Escopos aninhados e sobreamento de variável

Entrada:

```
opcao: 1 -> declarar z bool (escopo global)
opcao: 3 (entrar em novo escopo)
opcao: 1 -> declarar z string (escopo local – mesmo nome)
opcao: 2 -> buscar z
opcao: 5 (exibir pilha)
opcao: 4 (sair do escopo local)
opcao: 2 -> buscar z
```

Saída esperada:

```
[SUCESSO] Declarado: 'z' do tipo 'bool' no escopo atual.
```

```
[INFO] Novo escopo aberto.  
[SUCESSO] Declarado: 'z' do tipo 'string' no escopo atual.  
[SUCESSO] Variavel 'z' encontrada! Tipo: 'string'
```

```
--- ESTADO ATUAL DA PILHA DE ESCOPOS ---
```

```
Local (Nível 1): {'z': 'string'}
```

```
Global: {'z': 'bool'}
```

```
-----
```

```
[INFO] Escopo atual fechado (variaveis locais descartadas).
```

```
[SUCESSO] Variavel 'z' encontrada! Tipo: 'bool'
```

Resultado: PASSOU. O sombreamento funcionou corretamente e o escopo global foi restaurado após o fechamento.

Caso de Teste 4 — Busca de variável não declarada

Entrada:

```
opcao: 2  
variavel: naoExiste
```

Saída esperada:

```
[ERRO SEMANTICO] Variavel 'naoExiste' nao foi declarada em nenhum escopo visivel.
```

Resultado: PASSOU. O sistema reportou corretamente a ausência da variável em todos os escopos.

Caso de Teste 5 — Tentativa de fechar o escopo global

Entrada:

```
opcao: 4 (sair sem ter entrado em nenhum escopo local)
```

Saída esperada:

```
[ERRO] Nao e possivel fechar o escopo global.
```

Resultado: PASSOU. O sistema protegeu o escopo global contra fechamento acidental.

Todos os casos de teste apresentaram comportamento conforme o esperado, validando a corretude das operações de declaração, busca hierárquica, controle de escopos e detecção de erros semânticos.